



**Baptiste Cojean, Gatien Auffret, Aubin
Bergeret, Matéo Serandour et Jules
Renouard**

ESIR 2 INFO

Rapport de projet

Sommaire

Sommaire.....	2
1. Description technique.....	3
1.1 Définition Formelle et Spécification de la Grammaire.....	3
1.2 Architecture du compilateur et chaîne de compilation.....	3
1.3 Description de l'AST.....	4
1.4 Explication de l'analyse sémantique.....	6
1.5 Génération de code 3 adresses à partir de l'AST.....	8
1.6 Optimisation du code.....	10
1.7 Génération de code cible.....	11
1.7.1 Processus de traduction.....	11
1.7.2 Structure du fichier généré.....	11
1.8 Bibliothèque runtime de WHILE.....	11
1.8.1 Structure de données : L'Arbre Binaire.....	11
1.8.2 Gestion des primitives du langage.....	12
1.8.3 Entrées/Sortie et Conversion.....	12
1.8.4 Interface destinée à l'outil de traduction (Bridge).....	12
1.9 Détails du fonctionnement de Translator.java.....	12
1.9.1 Analyse et filtrage des entrées.....	13
1.9.2 Mécanisme de la pile (Stack) pour les arguments.....	13
1.9.3 Gestion dynamique des variables.....	13
1.9.4 Traduction des structures de contrôle et labels.....	14
1.9.5 Génération de la structure de sortie.....	14
2. Méthodologie de gestion de projet.....	14
2.1 Outils utilisés.....	14
2.2 Étapes de développement / découpage des tâches.....	14
3. Rapports individuels.....	15
4. Post-mortem.....	15
4.1 Ce qui a bien fonctionné et moins bien fonctionné.....	15
Ce qui a bien fonctionné.....	15
Ce qui a moins bien fonctionné.....	15
4.2 Avec plus de recul, comment ferions-nous maintenant ?.....	16
5. Annexes.....	16

1. Description technique

1.1 Définition Formelle et Spécification de la Grammaire

La grammaire du langage While a été définie formellement en utilisant ANTLR (ANother Tool for Language Recognition). Nous avons créé deux fichiers principaux :

- `While_grammar.g` : Contient les règles lexicales et syntaxiques basées sur les spécifications fournies. Cette grammaire définit la structure des programmes While, incluant les déclarations de fonctions, les commandes (affectations, structures de contrôle) et les expressions.
- `While_ast.g` : Spécifie la construction de l'arbre de syntaxe abstraite (AST) à partir de la grammaire, en utilisant les règles de réécriture d'ANTLR pour structurer hiérarchiquement les éléments du programme.

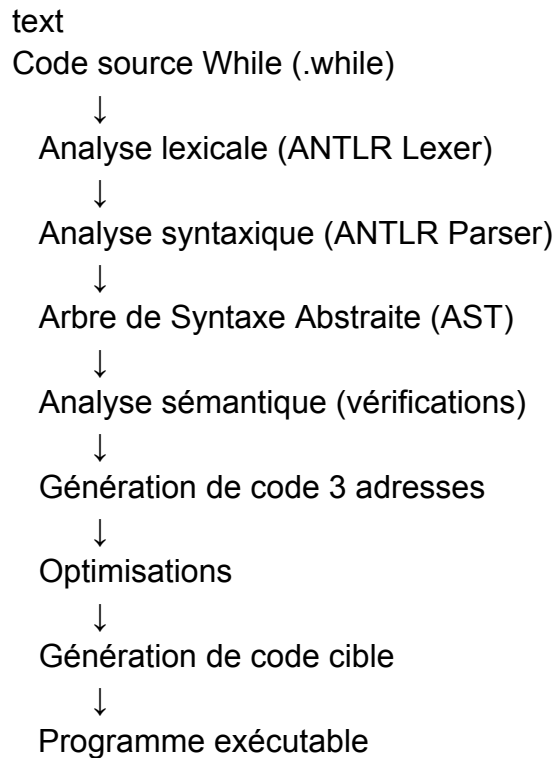
Nous verrons dans une autre partie la façon dont a été construit l'AST à partir du fichier `While_ast.g`. En ce qui concerne le fichier `While_grammar.g` nous l'avons alimenté à l'aide de la grammaire du langage While donné. Certains Non-terminaux ont dû être adaptés pour coïncider avec les spécificités de ANTLR. Quelques exemples des principaux Non-terminaux que nous avons défini:

- `function` : permettant la définition des fonctions
- `commands`: associer à chaque traitement au sein de la fonction
- `exprs` : permettant la déclaration des variables et l'appel à celle-ci ainsi que la vérification de condition.

Pour déterminer si la grammaire était correcte, nous avons défini une batterie de tests que nous utiliserons au fur et à mesure de l'avancée dans la chaîne de compilation.

1.2 Architecture du compilateur et chaîne de compilation

Notre compilateur suit une architecture modulaire en plusieurs passes, transformant progressivement le code source While en code exécutable :



Chaque phase est isolée et communique via des structures de données bien définies (AST, code intermédiaire), permettant une évolution et un débogage aisés.

1.3 Description de l'AST

L'objectif de départ est de passer d'un arbre relativement complexe qui affiche notamment les ponctuations, à un arbre plus lisible. Concrètement on souhaite passer du parse tree généré par la grammaire dans antlr à l'AST. Cet AST servira pour les étapes suivantes à savoir l'analyse sémantique et la génération de code.

Pour obtenir cet AST, nous utilisons plusieurs techniques complémentaires. Nous utilisons des opérateurs de construction : “^” qui transforme un token en racine de sous arbre et “!” qui permet d'exclure des tokens de l'arbre comme par exemple les parenthèses ou autre. Nous utilisons également des règles de réécriture qui nous permettent de restructurer des nœuds, par exemple pour une fonction nous créons un nœud virtuel FUNC qui regroupe le nom et la définition.

Nous utilisons aussi des tokens qui nous servent de balises sémantiques. Ils ne sont pas dans le code source mais servent de racine dans l'AST.

La structure global de notre AST est la suivante :

- PROG : la racine unique de l'arbre, elle a pour fils une liste de noeuds FUNC
- FUNC : représente une fonction, a pour fils :
 - SYMBOL : nom de la fonction
 - PARAM : ses paramètres d'entrée
 - BLOCK : le corp de la fonction

- **RESULT** : les variables de sortie

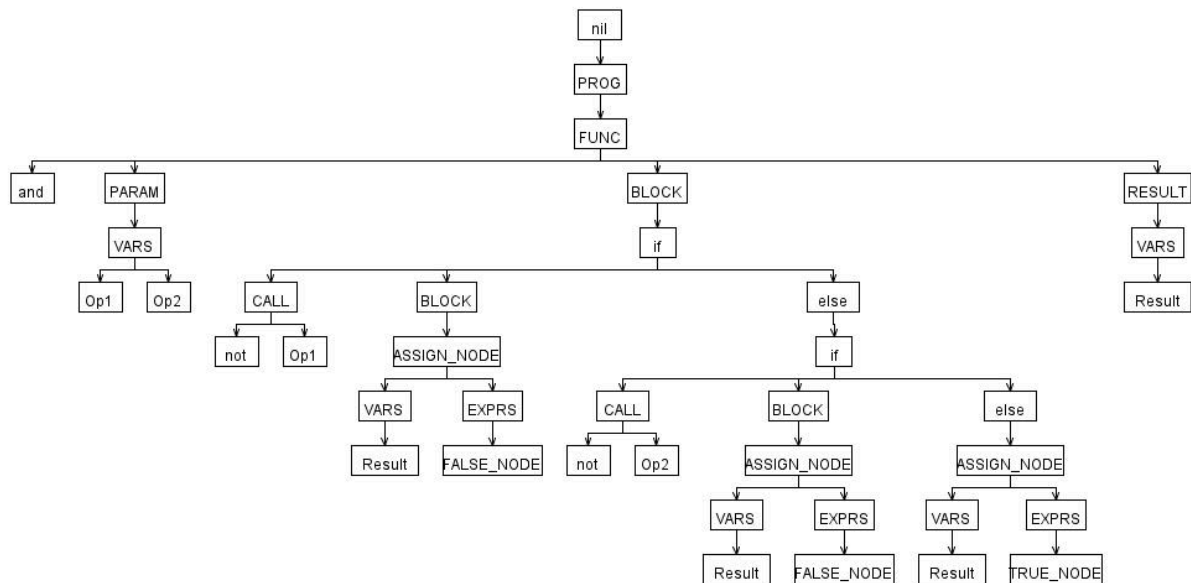
Le nœud **BLOCK** regroupe une séquence de commandes. L'assignation est utilisée via **ASSIGN_NODE** qui sépare les variables à gauche des expressions à droite afin de gérer nativement les affectations multiples. **BLOCK** regroupe également les structures de contrôle, à savoir : **IF**, **WHILE**, **FOR**, **FOREACH**. Ces nœuds conservent la condition comme premier fils et un nœud **BLOCK** comme second fils afin de représenter les instructions à exécuter. On supprime les mots clés de fermeture comme **FI** et **OD** car la structure de l'arbre définit la fin de la portée.

Les expressions sont simplifiées afin d'éviter les nœuds inutiles, **true** et **false** deviennent des nœuds explicites **TRUE_NODE** et **FALSE_NODE**, les opérations de liste deviennent des racines de sous-arbres avec leur argument en fils. Enfin **CALL** représente l'appel d'une fonction, liant le nom de la fonction aux expressions passées en arguments.

On prend pour exemple le et logique dont le code est le suivant :

```
// Logical and
function and :
read Op1, Op2
%
    if (not Op1) then
        Result := (false)
    else
        if (not Op2) then
            Result := (false)
        else
            Result := (true)
        fi
    fi
%
write Result
```

On génère son AST et obtien le résultat suivant :



1.4 Explication de l'analyse sémantique

Dans un premier temps, nous avons défini un fichier `While_ast.g` qui fait deux choses en même temps. La première chose, elle permet de faire la vérification syntaxique. En effet nous avons défini ce qui est autorisé ou non dans le langage, pour définir ces règles nous sommes aidés du PDF fournis.

La seconde chose que ce fichier nous permet de faire est la construction de l'AST. Il transforme le texte plat en un arbre structuré facile à parcourir.

Pour faire l'analyse sémantique, nous avons suivi une structure spécifique. Voici ce que nous avons fait :

1. Les structures de Données

Nous avons implémenté plusieurs fichiers qui définissent comment nous stockons les informations. Voici les différents fichiers : `Symbol.java`, `VariableSymbol.java`, `FunctionSymbol.java` et `SymbolTable.java`

2. Le cerveau

C'est dans le fichier suivant, que nous avons implémenté la plus grande partie des choses. Le fichier "`SemanticAnalyzer.java`" fonctionne en deux grands blocs :

- **Bloc1, scanFunctionSignature** : Cette première fonction nous permet de remplir la table des Symboles. En effet, la fonction parcourt l'AST pour extraire uniquement les noms des fonctions et leurs signatures (nom, nombre entrées et nombres de sorties). L'intérêt est que lors de la passe 2 nous saurons qu'une fonction existe même si elle est définie plus bas dans le fichier.
- **Bloc 2 analyzeFunctionBodies** : C'est dans cette fonction là, que la vérification à lieu. Cette fonction va donc, pour chaque fonction :

- **Déclare les entrées** : Il va donc ajouter les paramètres dans le scope local (mémoire de la fonction, pour éviter les collisions entre variable de même nom dans différentes fonctions)
- **Parcours les commandes** : Il analyse chaque instruction (IF, FOR, WHILE,...)
- **Vérifie les expressions** : Chaque fois que l'on utilise une variable, l'analyseur va vérifier à l'aide de la table local si elle est bien déclarée. Et à chaque appel de fonction (CALL) il vérifie le nombre d'arguments fournis avec la signature stockée.

3. Erreur rencontrer

Lors de l'analyse, une première version a été faite et nous avons testé sur les fonctions données dans la doc seulement il y avait un problème. En effet, étant donné que nous avons essayé seulement sur des fonctions sans erreurs, nous ne savions pas si notre programme était capable de les détecter. C'est pourquoi nous avons généré 4 fichiers pour tester les erreurs suivantes :

1. Test de Variable Non Défini

Nous avons tenté d'utiliser une variable X dans une expression alors qu'elle n'avait été ni passée en paramètre (read), ni assignée précédemment.

Résultat :

```
→ tlcwhile git:(Three_adresses) ✗ java -cp "bin:lib/antlr-3.5.3-complete-no-st3.jar" while_compiler.Main src/test/les_futurs_tests_ici.txt
=== COMPILATEUR WHILE ===
Compiling file: src/test/les_futurs_tests_ici.txt

--- Arbre de syntaxe abstraite (AST) ---
(PROG (FUNC testVarInconnue (PARAM (VARS A)) (BLOCK (ASSIGN_NODE (VARS Result) (EXPRS A)) ; (ASSIGN_NODE (VARS Result) (EXPRS (CONS X Result)))) (RESULT (VARS Result))))

--- Analyse Sémantique ---
testVarInconnue
A
Result
ErreurVariable non définie : X
```

2. Test de Fonction inconnue

Ce test consistait à appeler une fonction calculInexistant qui ne figure pas dans le code source.

Résultat :

```
→ tlcwhile git:(Three_adresses) ✗ java -cp "bin:lib/antlr-3.5.3-complete-no-st3.jar" while_compiler.Main src/test/les_futurs_tests_ici.txt
=== COMPILATEUR WHILE ===
Compiling file: src/test/les_futurs_tests_ici.txt

--- Arbre de syntaxe abstraite (AST) ---
(PROG (FUNC callInconnu (PARAM (VARS A)) (BLOCK (ASSIGN_NODE (VARS Result) (EXPRS (CALL calculInexistant A)))) (RESULT (VARS Result))))

--- Analyse Sémantique ---
callInconnu
A
Result
ErreurFonction inconnue : calculInexistant
```

3. Test de Signature Incorrecte

Nous avons appelé la fonction add (définie avec 2 paramètres) en lui passant 3 arguments (A B C).

Résultat

```
➔ tlewhile git:(three_adresses) ✗ java -cp "bin:lib/antlr-3.5.3-complete-no-st3.jar" while_compiler.Main src/test/les_futurs_tests_ici.txt
=== COMPILATEUR WHILE ===
Compiling file: src/test/les_futurs_tests_ici.txt

--- Arbre de syntaxe abstraite (AST) ---
(PROG (FUNC add (PARAM (VARS Op1 Op2)) (BLOCK (ASSIGN_NODE (VARS Result) (EXPRS (CONS Op1 Op2)))) (RESULT (VARS Result)))) (FUNC testSignature (PARAM (VARS A B C)) (BLOCK (ASSIGN_NODE (VARS Result) (EXPRS (CALL add A B C)))) (RESULT (VARS Result))))

--- Analyse Sémantique ---
add
testSignature
Op1
Op2
Result
A
B
C
Result
ErreurAppel à add: 3 args fournis, 2 attendus.
```

4. Test d'Appel imbriqué et récursivité

C'est le test le plus convaincant de la fiabilité de notre algorithme. Nous avons testé un appel complexe : `add (sub X) Z`. Ici, c'est la fonction imbriquée `sub` qui ne reçoit qu'un seul argument au lieu de deux.

Résultat

```
➔ tlewhile git:(three_adresses) ✗ java -cp "bin:lib/antlr-3.5.3-complete-no-st3.jar" while_compiler.Main src/test/les_futurs_tests_ici.txt
=== COMPILATEUR WHILE ===
Compiling file: src/test/les_futurs_tests_ici.txt

--- Arbre de syntaxe abstraite (AST) ---
(PROG (FUNC add (PARAM (VARS A B)) (BLOCK (ASSIGN_NODE (VARS R) (EXPRS A))) (RESULT (VARS R)))) (FUNC sub (PARAM (VARS A B)) (BLOCK (ASSIGN_NODE (VARS R) (EXPRS A))) (RESULT (VARS R)))) (FUNC testRecurisif (PARAM (VARS X Y Z)) (BLOCK (ASSIGN_NODE (VARS Result) (EXPRS (CALL add (CALL sub X Y) Z))) ; (ASSIGN_NODE (VARS Result) (EXPRS (CALL add (CALL sub X) Z)))) (RESULT (VARS Result))))

--- Analyse Sémantique ---
add
sub
testRecurisif
A
B
R
A
B
R
X
Y
Z
Result
ErreurAppel à sub: 1 args fournis, 2 attendus.
```

1.5 Génération de code 3 adresses à partir de l'AST

La génération de code 3 adresses (TAC - Three Address Code) transforme l'AST vérifié en un code intermédiaire simple, proche d'un assembleur bas niveau. Chaque instruction TAC a au plus trois opérandes et est de la forme :

Par exemple, l'expression `(add X Y)` pourrait devenir :

`t1 = X`

`t2 = Y`

`t3 = t1 + t2`

Ce code est équivalent à `t3 = X + Y`. Pour notre compilateur du langage While nous avons fait le choix de représenter de la manière la plus détaillé possible chaque instructions de la fonction. Pour réaliser des appels aux méthodes propres au langage nous avons fait le choix de les représenter de la même manière qu'en assembleur. En ce qui concerne le début/la fin de chaque fonction nous les avons représentés par "func begin name" "func end name" cela permet d'identifier le début

et la fin de notre fonction plus simplement et de mieux la délimiter. Les éléments ajoutés en paramètre sont "read" puis placés dans un registre. En ce qui concerne les boucles for, while, foreach les commandes sont gérés avec des goto que l'on pourra ensuite interpréter en c++.

Ainsi pour chaque appel à ces méthodes nous ajoutons dans la pile les éléments souhaités avec le mot clé param et on met le nombre d'élément après l'appel de la méthode. Ainsi cela permet de mieux gérer les appels à de nombreuses variable comme cela serait le cas avec la création d'une liste :

Result = (list Op1 Op2 Op3)

Donne:

param Op1

param Op2

param Op3

Result = LIST 3

Tout comme les méthodes spécifiques au langage While l'appel à des fonctions prenant des éléments en paramètre se fait aussi via le mot clé param:

Result = (add Op1 Op2)

Donne:

param Op1

param Op2

Result = add 2

Pour donner un exemple voici le type de fonction en entrée :

```
function mul :  
read Op1, Op2  
%  
    for Op1 do  
        Result := (add Result Op2)  
    od  
%  
write Result
```

Voici la sortie obtenue:

```

func begin mul
read Op1
read Op2
R1 = Op1
R2 = Op2
ifz Op1 goto break_for_L2
Result = nil
R3 = Result
  param Result
  param Op2
R3 = add 2
  param Op1
Op1 = TAIL 1
goto L1
break_for_L2
write Result
func end mul

```

Pour décrire cette fonction, on a la déclaration de celle-ci “func begin mul” les paramètres Op1 et Op2 sont read puis placés dans des registres (R1,R2). Ensuite on accède à la boucle for représenté par un ifz et vérifie que la variable Op1 n’est pas à nulle sinon elle retourne à la boucle break_for_L2. Si ce n’est pas le cas, on déclare la variable Result que l’on place dans le registre R3 puis on ajoute à la pile Result et Op2 pour ensuite appeler la fonction add sur ces 2 paramètres. On prend ensuite la queue de Op1 comme pour prendre (Op1 - 1). Quand on arrive à la fin de la boucle on renvoie Result (contenu dans le registre R3).

1.6 Optimisation du code

Le compilateur intègre un module d'optimisation agissant directement sur le code intermédiaire (TAC). L'objectif est de réduire le nombre de registres virtuels et d'instructions inutiles avant la traduction finale.

Tout d’abord le module repère les affectations simples de type $Ri = Op$. Il crée une table de remplacement pour substituer chaque utilisation ultérieure du registre Ri par la variable d'origine Op .

Une fois les substitutions effectuées, les instructions d'affectation vers les registres virtuels ($R1, R2, \dots$) maintenant inutiles sont supprimées de la liste d'instructions finale.

Ce traitement permet de générer un fichier .txt plus compact, facilitant ainsi la lecture du code C++ généré par le Translator.

1.7 Génération de code cible

La génération du code cible est la dernière étape du Backend. Elle a pour objectif de traduire le code 3 adresses en un code source C++ qui soit compilable, en s'appuyant sur la bibliothèque runtime pour la manipulation des structures de données.

1.7.1 Processus de traduction

Pour passer du C3A au code source nous avons suivi un processus très simple. Nous parcourons séquentiellement chaque instructions du code C3A et nous générons le code équivalent en C++.

Le mapping des instructions se fait de la manière suivante :

- **Affectations simples** : $R1 = Op1$ devient `Tree* R1 = Op1;`
- **Opérations primitives** : $R1 = hd\ R2$ est traduit par `Tree* R1 = hd(R2);`
- **Appels de fonctions** : Les instructions param sont accumulées pour générer un appel de fonction standard en C++.
- **Sauts et Étiquettes** : Les goto L1 et L1: du C3A sont traduits directement par les mots-clés goto et les labels de destination en C++.

1.7.2 Structure du fichier généré

Comme pour chaque fichier C++, notre code source de sortie doit posséder une structure précise pour son bon fonctionnement :

- **Inclusions** : Ajout de `#include "while_runtime.h"` pour accéder aux types et fonctions de base.
- **Déclarations de fonctions** : Prototype de toutes les fonctions WHILE pour permettre les appels récursifs ou croisés.
- **Corps des fonctions** : Traduction ligne à ligne du C3A.
- **Fonction main** : Une fonction principale est générée pour initialiser le runtime (`while_init()`), lire les entrées depuis la console, appeler la fonction WHILE principale et afficher le résultat final via `write_variable`.

1.8 Bibliothèque runtime de WHILE

Afin d'exécuter le générateur à partir des programmes WHILE, nous avons développé une bibliothèque runtime en C++. Cette bibliothèque nous est utile afin de manipuler les données du langage (arbres binaires) et assure l'interface en instructions de bas niveau (C3A) et l'exécution réelle.

1.8.1 Structure de données : L'Arbre Binaire

Dans ce langage, une seule structure de données est existante et c'est l'arbre binaire.

Nous avons implémenté cette structure via un type `Tree` composé de :

- **NodeType**: C'est un type "énuméré" (`NODE_NIL`, `NODE_CONS`, `NODE_SYMBOL`), ce qui nous permet d'identifier la nature de chaque nœud.
- **Pointeur car/cdr** : Pour les nœuds qui sont de type `CONS`, des pointeurs pointant vers les sous-arbres gauche et droit (`car/cdr`).
- **Symbol** : Une chaîne de caractère utilisée pour les feuilles de type symbole

1.8.2 Gestion des primitives du langage

Dans cette bibliothèque, nous avons aussi implémenté les fonctions primitives indispensables de `WHILE`, pour gérer les arbres :

- **Construction** : `cons(Tree* car, Tree* cdr)` crée un nouveau nœud liant deux arbres.
- **Accesseurs** : `hd(Tree* t)` (Head) et `tl(Tree* t)` (Tail) permettent de décomposer les arbres. Ils sont sécurisés : l'accès à la tête d'un atome ou d'un `NIL` renvoie systématiquement `NIL`.
- **Logique** : La fonction `is_true(Tree* t)` définit la sémantique booléenne du langage : tout ce qui est un `CONS` est considéré comme `VRAI`, tandis que `NIL` et les symboles sont considérés comme `FAUX`.

1.8.3 Entrées/Sortie et Conversion

Nous avons aussi implémenté plusieurs fonctions utiles notamment pour faciliter l'interaction avec l'utilisateur :

- **Entiers vers Arbres** : Puisque `WHILE` manipule souvent des entiers représentés par des listes de `NIL` (système unaire), nous avons inclus `int_to_tree` et `tree_to_int`.
- **Affichage (Pretty Printer)** : La fonction `pp(Tree* t)` permet d'afficher l'arbre de manière lisible sous forme de parenthèses (ex: `(cons nil (cons nil nil))`).

1.8.4 Interface destinée à l'outil de traduction (Bridge)

Afin d'optimiser la création de code à partir de notre compilateur Java, nous avons intégré une classe utilitaire `TreeNodeUtil`. Elle encapsule les appels système et facilite la conversion de listes d'éléments provenant du traducteur directement en structures d'arbres C++, garantissant ainsi une transition fluide entre le code à trois adresses et l'exécutable final.

1.9 Détails du fonctionnement de `Translator.java`

Cette fonction Translator.java est le coeur de notre projet. En effet, son rôle est de transformer le fichier C3A en un fichier source C++ valide. Voici une explication de notre fonction :

1.9.1 Analyse et filtrage des entrées

Dans un premier temps, le fichier C3A est passé au filtrage afin de supprimer les lignes inutiles de type, les commentaires (//) et les marqueurs de séparation (%). Cette phase de nettoyage permet de ne conserver que les instructions sémantiquement significatives pour la génération de code.

1.9.2 Mécanisme de la pile (Stack) pour les arguments

Pour gérer les paramètres donnés par le C3A, nous avons utilisé un système de PILE. Pour mieux expliquer je vais donner un exemple :

Étape	Ligne C3A lue	Action de la Pile (javaStack)	État de la pile (Sommet à droite)	Code C++ généré
1	param B	push("B")	[B]	(Rien)
2	param A	push("A")	[B, A]	(Rien)
3	R = CONS 2	op2 = pop() op1 = pop()	[] (vide)	R = TreeNode::CONS(A, B);

1.9.3 Gestion dynamique des variables

Le traducteur créer une liste (tacVar) pour suivre les variables déjà déclarées dans le corps de la fonction C++.

- Si une variable de destination (à gauche du =) n'a jamais été rencontrée, le traducteur génère alors une déclaration de type (TreeNode* var = ...).
- Si elle existe déjà, il génère une simple affectation (var = ...). Cela permet de produire un code C++ qui respecte le typage statique tout en gérant les variables temporaires générées par le frontend.

1.9.4 Traduction des structures de contrôle et labels

En C3A, nous avons plusieurs types de structure de contrôle et labels, que nous avons dû traduire. C'est pourquoi lorsque que nous avons une ligne possédant un certain mot clé nous le traduisons en C++. Exemple :

- **GOTO** : Traduit directement par l'instruction goto du C++.
- **IF** : La condition IF condition GOTO label est traduite par un test de nullité (if (condition != nullptr)). Conformément à la sémantique de WHILE, un pointeur non nul est considéré comme vrai.
- **Labels** : Les étiquettes (ex: L1:) sont recopiées textuellement, servant de points d'ancrage pour les branchements.

1.9.5 Génération de la structure de sortie

Pour finir et rendre le code source, on assemble toutes les parties entre elles :

- **Header** : Inclusion des bibliothèques nécessaires (while_runtime.h, vector, iostream).
- **Signature** : Construction du nom de la fonction et de sa liste de paramètres à partir des instructions func begin et read.
- **Corps** : Insertion des instructions générées dans le body.
- **Main** : Génération d'un point d'entrée main (actuellement minimal) pour permettre la compilation du fichier.

2. Méthodologie de gestion de projet

2.1 Outils utilisés

Pour la communication nous avons utilisé discord, très pratique pour les groupes, très pratique pour partager des images, du texte ou pour faire des appels groupés.

Pour le partage des fichiers nous avons mis en place un Git auquel chacun des membres avait accès. Dès l'ajout d'une modification via un push ou un merge nous avons également pris l'habitude de le notifier sur le discord afin que chacun aie une idée de l'avancé de chacun, cela permettait aussi de rappeler de pull les dernières modifications avant de continuer le projet pour éviter aux maximum les conflits.

2.2 Étapes de développement / découpage des tâches

En premier lieu, nous avons dû réaliser la grammaire du langage while, essentielle pour la suite du projet, pendant ce temps, les membres du groupe non assignés a cette tâche devaient se renseigner sur les étapes suivantes afin de ne pas perdre de temps. Nous

avons ensuite dû réaliser l'AST lui aussi indispensable pour la suite du projet. Cet AST nous a permis de réaliser l'analyse sémantique incluant la table des symboles ainsi que la validation du programme. Il nous a aussi permis de générer le code 3 adresses du langage while utile pour le backend que nous avons réalisé ensuite. Pour finir nous avons réalisé un script permettant d'enchaîner le compilateur While avec le compilateur du langage cible de manière à générer un exécutable.

3. Rapports individuels

Baptiste Cojean : Réalisation partielle de la grammaire du langage while, réalisation complète de l'AST, réalisation complète de l'optimisation et réalisation en grande partie du compte rendu.

Gatien Auffret : Réalisation de l'analyse sémantique avec Matéo, Création des test, Réalisation de la bibliothèque runtime. Réalisation du script du compilateur

Aubin Bergeret : Réalisation de l'optimisation du code 3 adresses, du compte rendu et structure globale du dépôt Git (Documentation)

Matéo Serandour : Réalisation de l'analyse sémantique avec Gatien, Réalisation du Backend notamment faisant la fonction Translator avec Jules, création de la bibliothèque while, correction de bug et réalisation du compte rendu.

Jules Renouard :

4. Post-mortem

4.1 Ce qui a bien fonctionné et moins bien fonctionné

Ce qui a bien fonctionné

La communication était bonne et la répartition des tâches a été globalement bonne grâce à cette communication. Également, nous avons réparti la plupart des tâches individuellement, cependant les tâches plus importantes, que ce soit de part leur nécessité pour avancer ou simplement de part leur complexité ont été assignés par binôme. Cela nous a permis de ne pas rester bloqué trop longtemps tout en fournissant un résultat fonctionnel.

Ce qui a moins bien fonctionné

Nous nous y sommes pris trop tard, ce qui ne nous a pas permis d'avoir un projet parfaitement abouti. Nous avons notamment sous-estimé la complexité de l'étape de génération de code et de l'optimisation. Nous avons passé beaucoup de temps sur

la grammaire au début, ce qui a créé un manque de temps en fin de projet pour l'implémentation des fonctionnalités avancées.

4.2 Avec plus de recul, comment ferions-nous maintenant ?

Avec les nouvelles connaissances en gestion de projet acquises durant ce semestre, si c'était à refaire nous les utiliserions plus. Notamment l'utilisation de méthode agile avec la mise en place de différents sprint afin d'être moins pris de court par l'échéance finale. Nous mettrions aussi en plus plus de tests, unitaires par exemple.

5. Annexes